

How to use the python coulter counter fitting scripts

For unimodal traces

How do the scripts work?

- Coulter counter quantifies starch granule volume – we collect with logarithmic bin spacing. Logarithmic bin spacing means that bins at smaller diameters are narrower than those at larger diameters, so peak heights must be normalised to account for this
- Script normalises for logarithmic bin spacing. If the data has been collected with linear bins this step will have no effect and the script can still be used.
- Curve fitting is carried out with `optimise.curve_fit` function from the `scipy 1.11.1` package. It will use provided initial values as a starting point for curve fitting and will iterate overall lots and lots of fittings until it produces the optimum fitting.
- After curve fitting, the data and the fitted curves are divided by the sum of the raw volume (%) so that the total height of the data sums to 100%. This is a simple stretch and results in no changes to the overall fitting or calculated parameters. This ensures the units of the y-axis is in line with expectations, given the Coulter counter output.

What fittings does the script use and what does it produce?

- The script fits 3 different traces to unimodal coulter counter traces:
 - 1) A normal curve
 - 2) A lognormal curve
- It compares the different fittings using total uncertainty and a standard error of regression test (S).
- It produces estimates of mean granule diameter, the variance of the granule peak (a measure of how wide the peak is)

What are the scripts?

2) `Batch_running_of_coulter_counter_script_unimodal` – this script will take each coulter counter csv file in turn and feed them into the `coulter_counter_fitting_script`. It also saves all the parameters from the different fittings and produces summary excel sheets (see later for info about outputs)

 `Batch_running_of_coulter_counter_fitting_script_unimodal.ipynb`

 `coulter_counter_fitting_script_unimodal.ipynb`

 `Running script - unimodal distributions.ipynb`

1) `Running script – unimodal distributions` – this script is used to change our working directory and submit the other two scripts

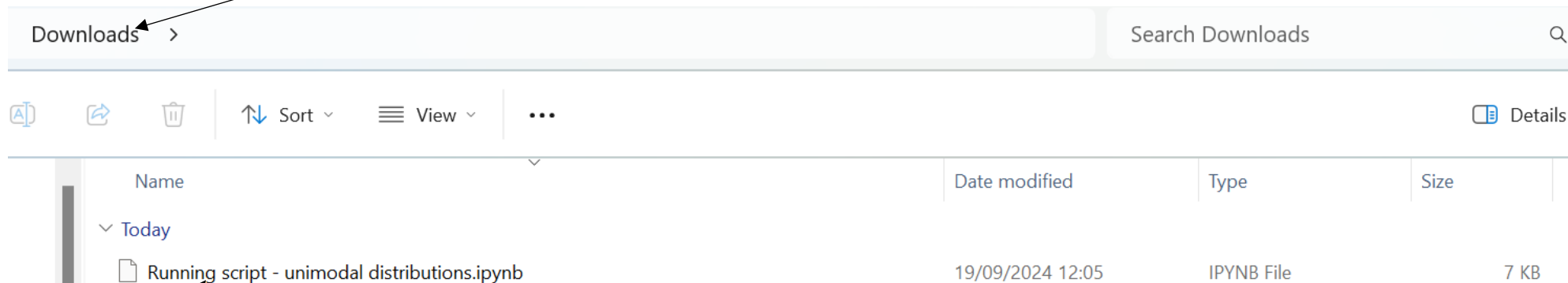
3) `coulter_counter_fitting_script_unimodal` – this script does the actual fittings. It takes one input file and fits the double normal, double lognormal and lognormal-normal curves to the data. It produces a PDF with graphs of the 3 different fittings and a comparison of the 3.

Why can't we just use the `coulter_counter_fitting_script` by itself? Technically we could but in order to get excel sheets with all the parameters in we need to use script 2 (`batch_running_of_coulter_counter_script`). It also means that if we have multiple coulter counter files to analyse at once we only need to submit the script once and not for every input file. The running script also makes it easier so that we can tell the computer where are input files are (so that we don't have to copy them into our Jupyter lab directory – see later for info about Jupyter lab).

How should my data be organised?

I always place the Running script (only this has to be here) in my Downloads as this is easy to navigate to using Jupyter notebook (see later)

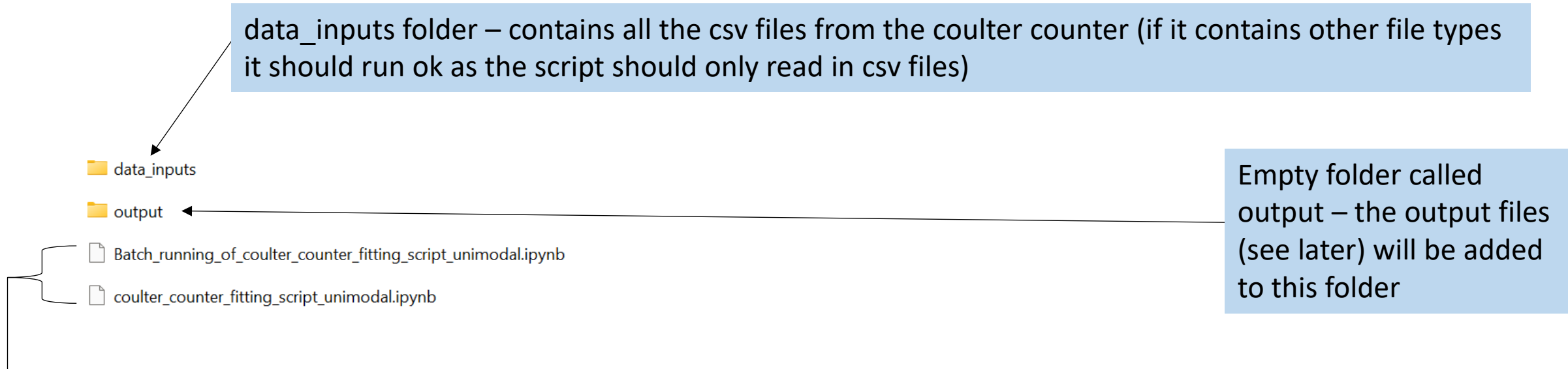
Download folder on your computer



Only the Running script needs to be here

How should my data be organised?

Everything else can be in a different folder, this can be anywhere:



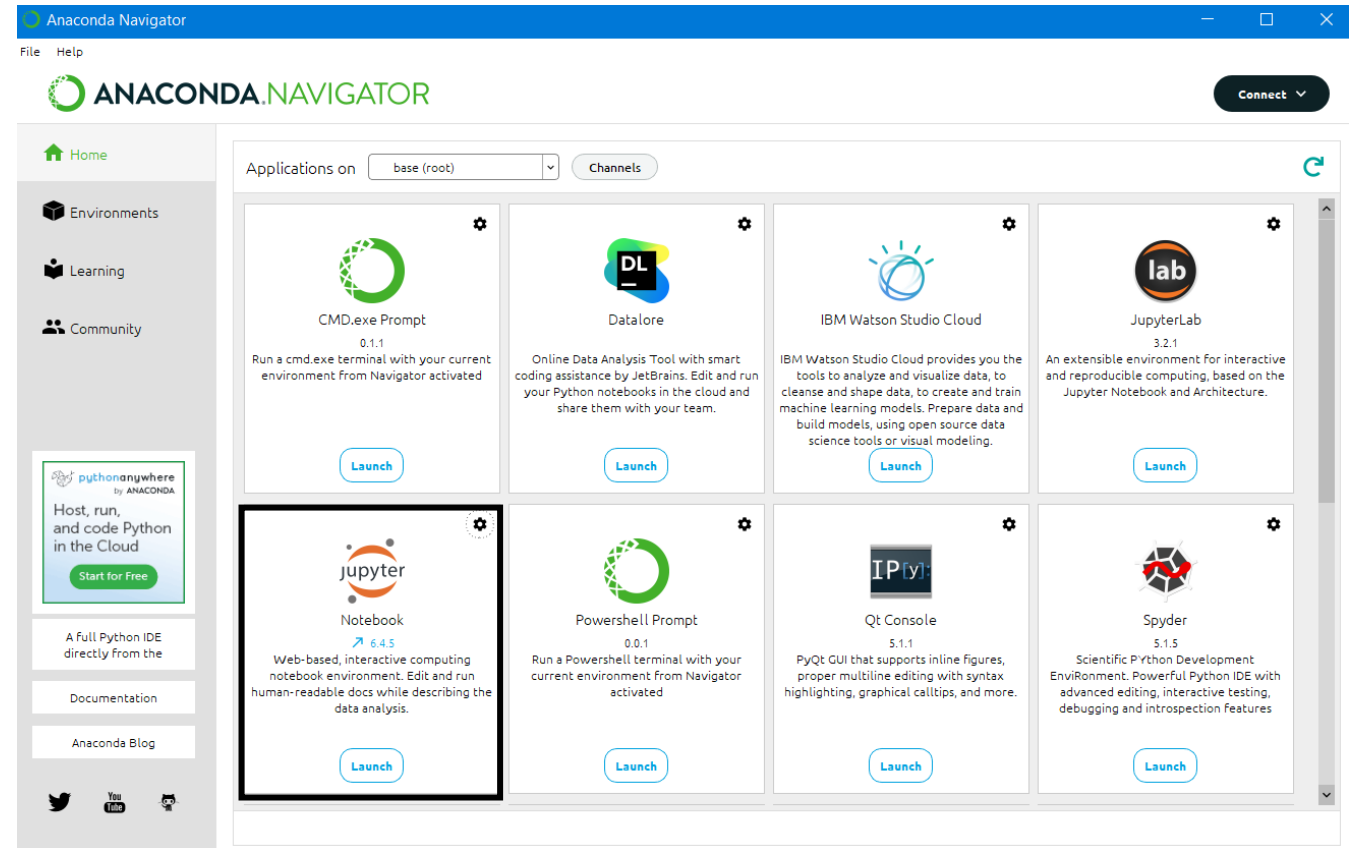
The batch_running_of_coulter_counter_fitting_script_unimodal and the coulter_counter_fitting_script_unimodal

The folder which is organised like this is your **working directory**, take note of the name of this folder and where it is (the full path) as you'll need it

How do I run the script?

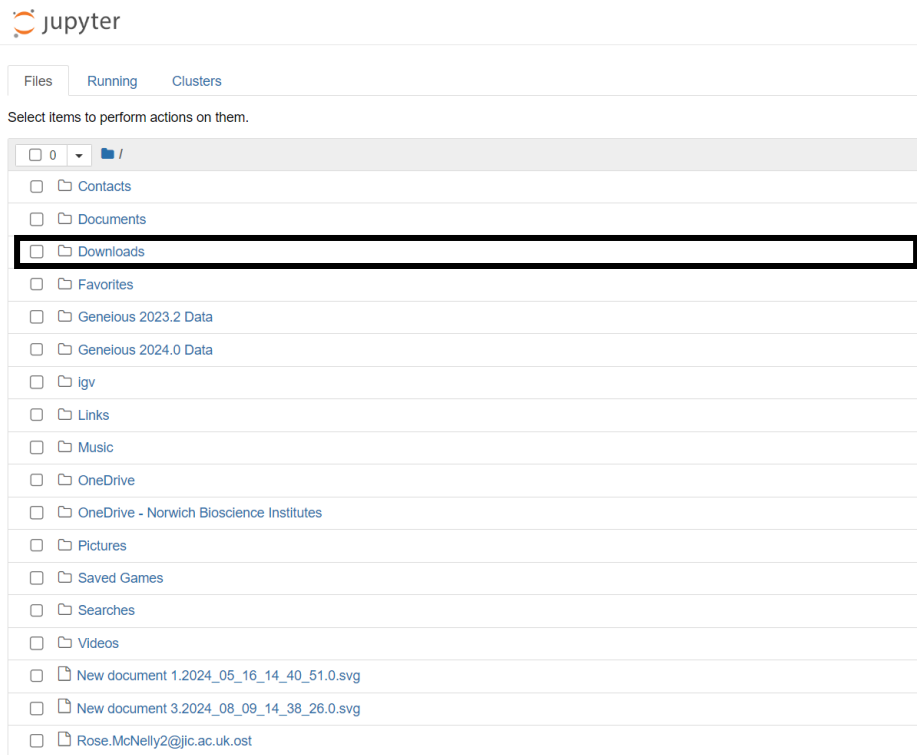
1) Download Anaconda Navigator

(<https://docs.anaconda.com/anaconda/install/>) and open Jupyter Notebook (this will open in an internet browser)

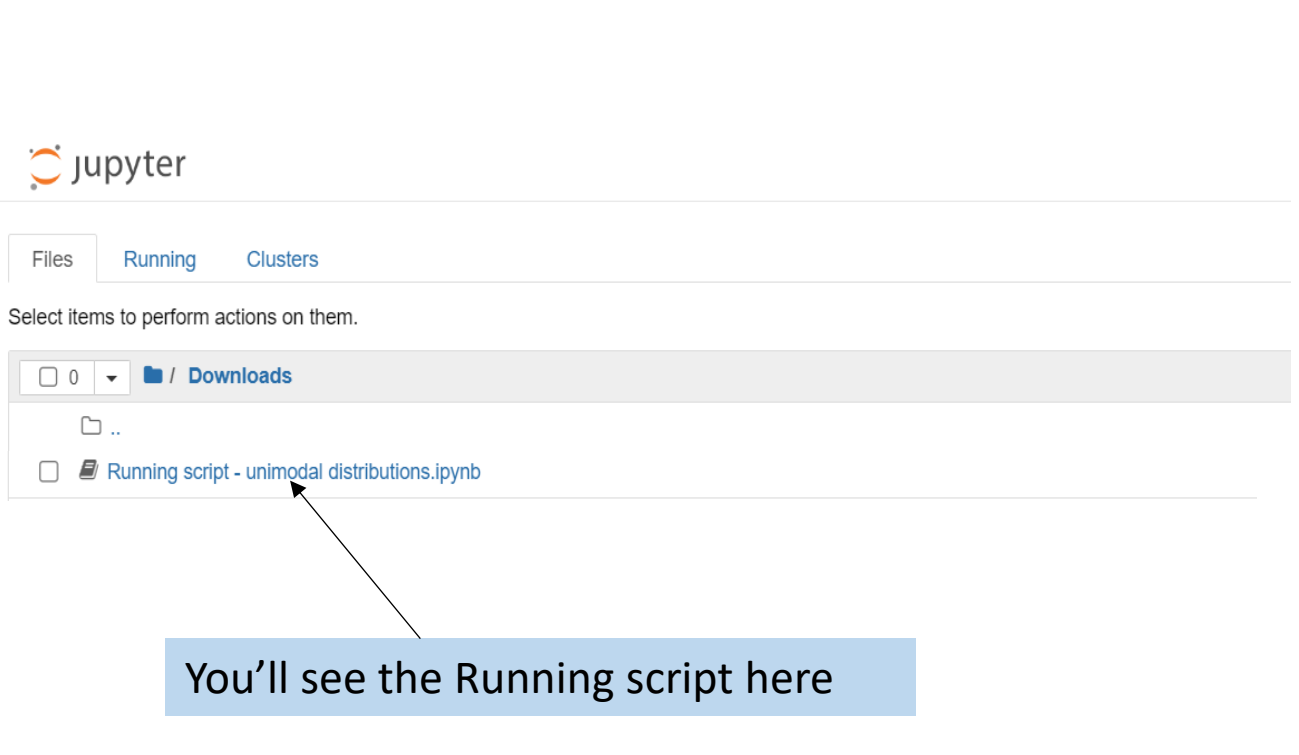


How do I run the script?

2) Navigate to your Downloads



The left screenshot shows the JupyterLab 'Files' view with the root directory selected. The 'Downloads' folder is highlighted with a black rectangle. An arrow points from this rectangle to the right screenshot.



The right screenshot shows the JupyterLab 'Files' view with the 'Downloads' folder selected. The file 'Running script - unimodal distributions.ipynb' is listed. A blue callout box with an arrow points to this file.

You'll see the Running script here

3) Open the Running script

How do I run the script?

```
In [ ]: #Loading libraries and packages
import os
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import matplotlib.ticker as mtick
import scipy.optimize
import scipy.stats
import scipy.integrate as integrate
import scipy.integrate as special
import pandas as pd
import linecache
from statistics import mean
import matplotlib.font_manager as font_manager
plt.style.use('ggplot') # use ggplot style for graps
```

Chunk 1

```
In [ ]: #Changing working directory
#Note - in your working directory you need folders called:
#1)data_inputs - which contains all the csv files from the coulter counter
#2)output - which is empty which your files will be placed in

os.chdir("FILL IN DATA LOCATION WITH FORWARD SLASHES")
#example below:
#os.chdir("C:/Users/mcnellro/OneDrive - Norwich Bioscience Institutes/59. Rose Bim
os.getcwd()
```

Chunk 2

4) Run the first chunk (shown as a grey box), by clicking in the chunk and pressing Ctrl and Enter.

If it has run successfully a small number should appear in the square brackets on the left of the chunk

How do I run the script?

```
In [ ]: #Loading libraries and packages
import os
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import matplotlib.ticker as mtick
import scipy.optimize
import scipy.stats
import scipy.integrate as integrate
import scipy.integrate as special
import pandas as pd
import linecache
from statistics import mean
import matplotlib.font_manager as font_manager
plt.style.use('ggplot') # use ggplot style for graphs
```

Chunk 1

```
In [ ]: #Changing working directory
#Note - in your working directory you need folders called:
#1)data_inputs - which contains all the csv files from the coulter counter
#2)output - which is empty which your files will be placed in

os.chdir("FILL IN DATA LOCATION WITH FORWARD SLASHES")
#example below:
#os.chdir("C:/Users/mcnellro/OneDrive - Norwich Bioscience Institutes/59. Rose Bim
os.getcwd()
```

Chunk 2

5) In chunk 2, change your working directory to the location which contains the other scripts, your data_inputs and output folder. You need to put in the full path, not just the folder name, and put it within quotation marks, e.g. "C:/Working_Directory_Folder/"

Run the chunk, if it has worked then it should print the working directory folder path directly underneath the chunk.

How do I run the script?

6) Do you want to change any of the optional parameters?

The optional parameters are:

- `x_value_to_plot_to` = default 50 - this is maximum value plotted on the x axis, for the 70 μm aperture 50 is enough, but should be increased if a larger aperture is used so that the data are not truncated
- `display_initial_parameter_graphs` = default 'FALSE', options are 'TRUE' or 'FALSE', if true then plots of the initial parameters will be saved, these can be useful if the fits are bad and you want to alter the initial parameters before rerunning the fitting
- initial parameters used for the fittings can be changed individually – I would normally run with the defaults and only change these if the fittings are bad

How do I run the script?

6) Do you want to change any of the optional parameters? IF NO go straight to step 7.

IF YES:

Example: here I want to display the initial parameter graphs. I have removed the # from the front of the row and changed 'FALSE' to 'TRUE'

```
In [ ]: #Run script

#This will execute the Batch_running_of_coulter_counter_fitting_script_one_curve_only which will then run the coulter_counter_fit
#This will produce an output folder containing:
#1) PDF files for each individual trace which compare the fittings from all 3 models
#2) Excel file called 'Parameters_from_fittings' which contains all the parameters from the different models
#3) Excel file called 'Samples_which_fitted_successfully_and_samples_which_failed' which shows if the models couldn't be fitt
#4) Excel file called 'Samples_where_the_script_failed' which shows where there was an error during data import, e.g. maybe t

#Run script

#optional parameters:
#if you want to use take out the # and change the value to what you desire
#the script will run fine if not used so if you don't want to use any leave them all with # at front

#x_value_to_plot_to = 50 #default = 50, this is maximum value plotted on the x axis, for the 70 um aperture 50 is enough, but she

#display_initial_parameter_graphs = 'FALSE' #default = 'FALSE', options are 'TRUE' or 'FALSE', if true then plots of the initial

#normal_initial_A = 100 #default = 100
#normal_initial_mu = 5 #default = 5
#normal_initial_sigma = 2 #default = 2

#Lognormal_initial_A = 200 #default = 200
#Lognormal_initial_mu = 2 #default = 2
#Lognormal_initial_sigma = 1 #default = 1

%run Batch running of coulter counter fitting script unimodal.ipynb #This line should NOT have a # in front ever!
```

Chunk 3

```
In [ ]: #Run script

#This will execute the Batch_running_of_coulter_counter_fitting_script_one_curve_only which will then run the coulter_counter_fit
#This will produce an output folder containing:
#1) PDF files for each individual trace which compare the fittings from all 3 models
#2) Excel file called 'Parameters_from_fittings' which contains all the parameters from the different models
#3) Excel file called 'Samples_which_fitted_successfully_and_samples_which_failed' which shows if the models couldn't be fitt
#4) Excel file called 'Samples_where_the_script_failed' which shows where there was an error during data import, e.g. maybe t

#Run script

#optional parameters:
#if you want to use take out the # and change the value to what you desire
#the script will run fine if not used so if you don't want to use any leave them all with # at front

#x_value_to_plot_to = 50 #default = 50, this is maximum value plotted on the x axis, for the 70 um aperture 50 is enough, but she

display_initial_parameter_graphs = 'TRUE' #default = 'FALSE', options are 'TRUE' or 'FALSE', if true then plots of the initial

#normal_initial_A = 100 #default = 100
#normal_initial_mu = 5 #default = 5
#normal_initial_sigma = 2 #default = 2

#Lognormal_initial_A = 200 #default = 200
#Lognormal_initial_mu = 2 #default = 2
#Lognormal_initial_sigma = 1 #default = 1

%run Batch_running_of_coulter_counter_fitting_script_unimodal.ipynb #This line should NOT have a # in front ever!
```

Chunk 3

Chunk 3

Follow the same procedure for changing any other parameters. If you want to run with the defaults in the future you will need to re-add the # to the front of the line

How do I run the script?

```
In [ ]: #Run script

#This will execute the Batch_running_of_coulter_counter_fitting_script_one_curve_only which will then run the coulter_counter_fit
#This will produce an output folder containing:
#1) PDF files for each individual trace which compare the fittings from all 3 models
#2) Excel file called 'Parameters_from_fittings' which contains all the parameters from the different models
#3) Excel file called 'Samples_which_fitted_successfully_and_samples_which_failed' which shows if the models couldn't be fitt
#4) Excel file called 'Samples_where_the_script_failed' which shows where there was an error during data import, e.g. maybe t

#Run script

#optional parameters:
#if you want to use take out the # and change the value to what you desire
#the script will run fine if not used so if you don't want to use any Leave them all with # at front

#x_value_to_plot_to = 50 #default = 50, this is maximum value plotted on the x axis, for the 70 um aperture 50 is enough, but sho

display_initial_parameter_graphs = 'TRUE' #default = 'FALSE', options are 'TRUE' or 'FALSE', if true then plots of the initial po

#normal_initial_A = 100 #default = 100
#normal_initial_mu = 5 #default = 5
#normal_initial_sigma = 2 #default = 2

#lognormal_initial_A = 200 #default = 200
#lognormal_initial_mu = 2 #default = 2
#lognormal_initial_sigma = 1 #default = 1

%run Batch_running_of_coulter_counter_fitting_script_unimodal.ipynb #This line should NOT have a # in front ever!
```

Chunk 3

7) Run chunk 3 – this submits the Batch_running_of_coulter_counter_script_unimodal which itself will submit the coulter_counter_fitting_script_unimodal (you won't have to do anything)

You'll start to see the outputs appear in the output folder

What are the outputs?

Only produced because I set `display_initial_parameter_graphs = 'TRUE'` (step 6), if not you will not see this. Contains PDF files with plots using the initial parameters

 `Graphs_of_initial_parameters`

Contain graphs of all the different fittings

 `PDFs`

 `Mathematical_values_to_reproduce_fittings` - these are not the means and variance values

Mathematical_values_to_reproduce_fittings - these are not the means and variance values – contains all fitting parameters from the optimised fitting, along with uncertainties for each fitting parameter, to allow users to re-plot the mathematical curves if required.

 `Plotted standard error of regressions and uncertainty`

Plotted standard error of regressions and uncertainty – a graph comparing the standard error of regressions and uncertainties for the different fittings

 `Samples_where_the_script_failed`

Samples_where_the_script_failed – contains a list of filenames where the script failed (most likely during data import), if a filename appears in this spreadsheet it could be because the coulter counter CSV file is corrupted or not in the correct format

 `Samples_which_fitted_successfully_and_samples_which_failed`

Samples_which_fitted_successful_and_samples_which_failed – contains a list of filenames for each of the different fitting methods for samples which worked and samples where the fitting failed. The fitting might have failed if the data is not unimodal in shape.

 `Starch_parameters_from_fittings`

Starch_parameters_from_fittings – contains all the parameters from the 3 different fittings and also contains the uncertainties and the standard error of regression

See subsequent pages for more info about all the different outputs

Variance is the variance of the peak
(a measure of the peak width).

Starch_parameters_from_fittings

Filename

Normal fitting

Lognormal fitting

	A	B	C	D	E	F	G	H	I	J
		Sample_name	Granule diameter normal fitting	Variance for the normal fitting	Uncertainty for the normal fitting	Standard error of regression for the normal fitting	Granule diameter lognormal fitting	Variance for the lognormal fitting	Uncertainty for the lognormal fitting	Standard error of regression for the lognormal fitting
1										
2	0	Maize_1_31 May 2024_65.#m4	16.09167	21.2844	0.650424	0.001522	17.41275	30.40084	1.499199	0.003263
3	1	Maize_2_31 May 2024_66.#m4	16.06012	21.56101	0.592246	0.001382	17.41898	31.17293	1.437165	0.003107
4	2	Maize_3_31 May 2024_68.#m4	16.03593	21.17072	0.60741	0.001426	17.35647	30.17483	1.444316	0.003154
5	3	Maize_4_31 May 2024_69.#m4	16.08416	21.50905	0.584463	0.001364	17.42352	30.92075	1.445472	0.003131

Each row contains
the parameters
from a different
input file

Standard error of regression is a form of R^2 for non-linear models. It measures how well the predicted model fits the data. Higher standard errors of regression means that the data deviates from the model more. So **we want the standard error of regression to be as low as possible**. Uncertainty is a measure of how sure we can be about the parameters. Higher uncertainties mean that the parameters in the models could take lots of different values and still result in the same fitting. Therefore **we want the uncertainty to be as low as possible** as this means we are more certain that the model parameters are 'correct'. The **standard error of regression should be used to drive model selection**.

Mathematical_values_to_reproduce_fittings - these are not the means and variance values

Filename		Normal fitting						Lognormal fitting					
B	C	D	E	F	G	H	I	J	K	L	M	N	
Sample_name	Normal fitting A	Normal fitting A uncertainty	Normal fitting mu	Normal fitting mu uncertainty	Normal fitting sigma	Normal fitting sigma uncertainty	Lognormal fitting A	Lognormal fitting A uncertainty	Lognormal fitting mu	Lognormal fitting mu uncertainty	Lognormal fitting sigma	Lognormal fitting sigma uncertainty	
e_1_31 May 2024_65.#m4	99.99276	0.588623	16.09167	0.032204	4.613502	0.029596	102.772	1.489202	2.809427	0.005402	0.309114	0.004595	
e_2_31 May 2024_66.#m4	99.45688	0.535386	16.06012	0.029657	4.643384	0.027204	102.5389	1.427456	2.808662	0.005254	0.312724	0.004455	
e_3_31 May 2024_68.#m4	99.53978	0.549601	16.03593	0.030127	4.601165	0.027682	102.3611	1.43465	2.806234	0.005223	0.308968	0.004443	
e_4_31 May 2024_69.#m4	99.81198	0.528586	16.08416	0.029135	4.637785	0.026742	102.7407	1.435762	2.809324	0.005251	0.311439	0.004458	

Each row contains the parameters from a different input file

These values should only be used if you want to replot the fitting.
It also contains the uncertainty for each individual parameter.

Samples_where_the_script_failed

	A	B
1		Samples where the script failed - most likely during data import, so check coulter_counter file
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		

If any of the input files failed to load into the script properly they will be listed in this column.

Here this column is empty so all the files were inputted correctly.

NOTE – this does NOT mean that the fitting won't work as it has failed before the fitting has been attempted.

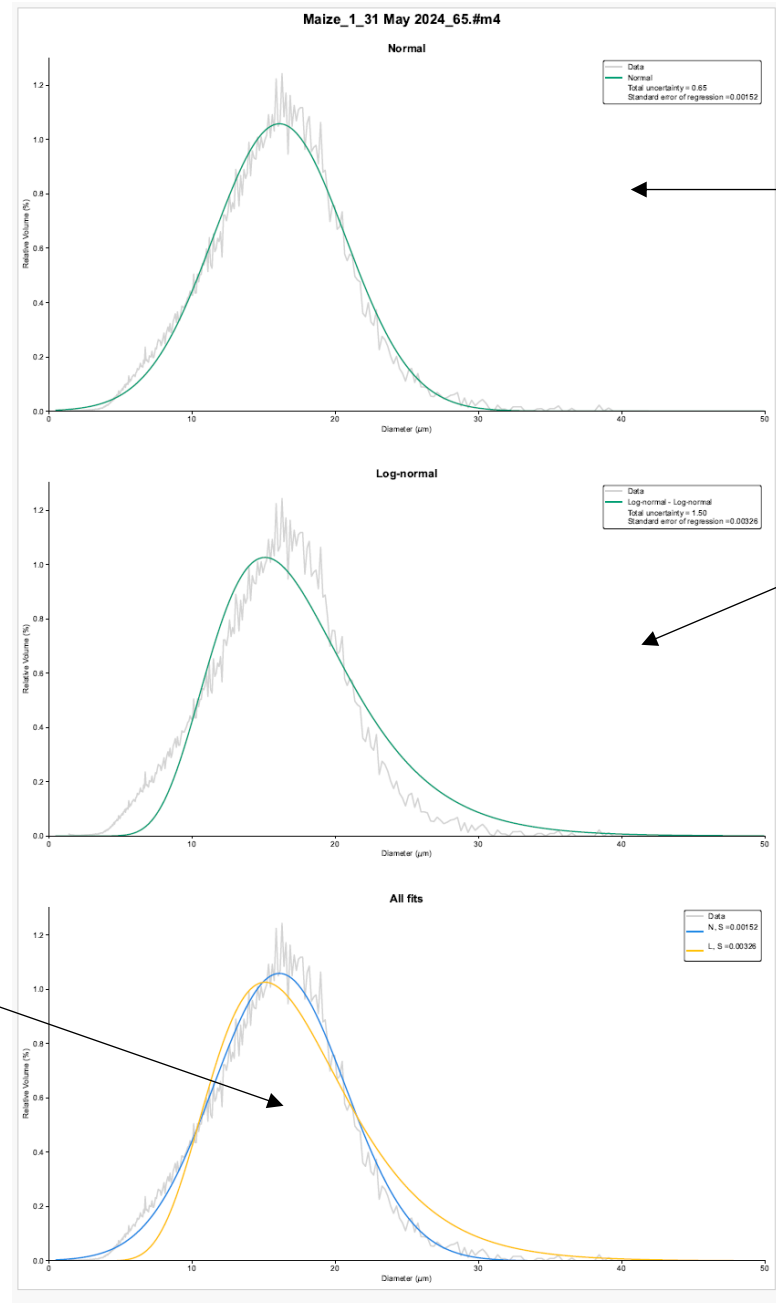
Samples_which_fitted_successful_and_samples_which_failed

	A	B	C	D	E
		Successful samples for the normal fitting	Failed samples for the normal fitting	Successful samples for the lognormal fitting	Failed samples for the lognormal fitting
1					
2	0	Maize_1_31 May 2024_65.#m4	All_successful	Maize_1_31 May 2024_65.#m4	All_successful
3	1	Maize_2_31 May 2024_66.#m4		Maize_2_31 May 2024_66.#m4	
4	2	Maize_3_31 May 2024_68.#m4		Maize_3_31 May 2024_68.#m4	
5	3	Maize_4_31 May 2024_69.#m4		Maize_4_31 May 2024_69.#m4	
6					

Tells us if the fitting failed for the normal (column C) or lognormal (column E) fittings. If any of the fittings failed then the filename will be listed in these columns. This means that the model couldn't be fitted to the data most likely as it significantly deviates from being unimodally distributed. It could be the case that one model might not fit but the other two might.

What should I do if the fitting is failing quite often? You could adjust the initial parameter values to fit your data more closely (see step 6 – adjust optional parameter values). If this does not help this could suggest a unimodal distribution is not an appropriate model for your data and you might need to use something else, e.g. is it more bimodal?

PDF files



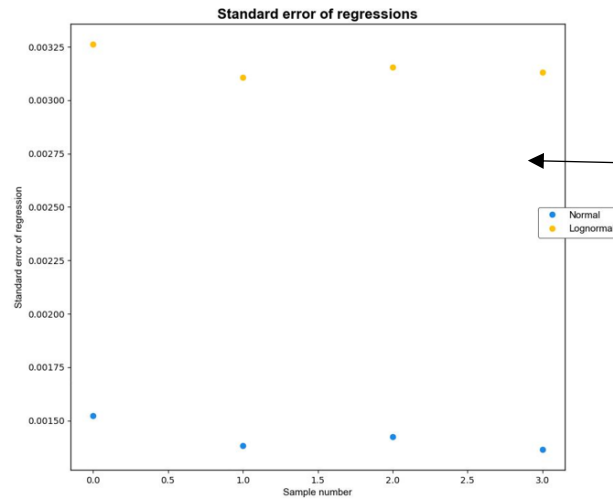
Normal fitting

Lognormal fitting

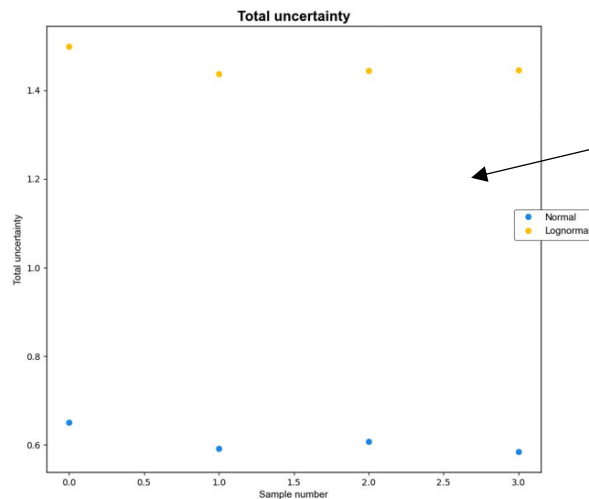
Comparison of normal (blue), lognormal (yellow) fittings with the raw data (grey) and the standard error of regression for each model

Grey = coulter counter data,,
turquoise = fitting. Also
includes the uncertainty and
standard error of regression
measurements.

Plotted standard error of regressions and uncertainty



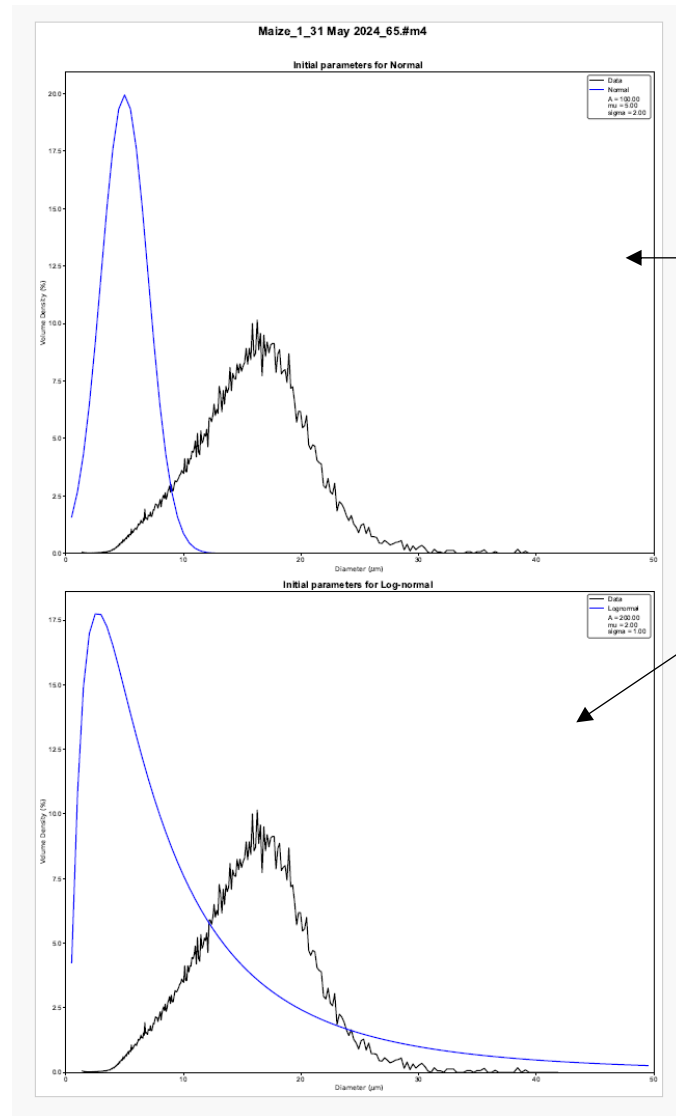
Comparison of normal (blue) and lognormal (yellow) standard error of regression values



Comparison of normal (blue) and lognormal uncertainties

To convert sample number to sample name look at columns A and B of Starch_parameters_from_fittings

Optional output: Graphs of initial parameters



Normal fitting

Lognormal fitting

Black = coulter counter data, blue = unoptimized fitting using the initial parameter values

Shows that the curve from the initial parameters does not have to be a good match as the fitting is still good (two pages previously)